

Ryan Ramdihal

EEL 4742

Lab 8: Analog to Digital Converter

Introduction:

In this lab, we will be interfacing with a 2-dimensional joystick using the Analog-to-Digital Converter (ADC) module. Using the ADC12_B module which uses a 12-bit resolution, which converts to 14 cycles as read from the data sheet. Our primary objective is to use the ADC to capture data based upon the joystick's movement. Followed by reconfiguring the ADC to capture the joysticks X and Y coordinates, using the function provided as a basis, and looking into the User's guide and Data sheet.

8.1:

The approach for this lab is to configure the ADC based on the comments provided in the lab manual to read the movements of the joystick. We also configure ACLK to cause an interrupt to blink the red LED when the program is running. In main, we use an infinite loop to poll for input from the joystick connected to the ADC. While the ADC12BUSY flag is active, it is locked in a loop until the conversion is complete. The data is written to UART. In the ADC function, we configure the cycles to 32 based upon the information given in the lab manual and the equation I produced. $3 \times 10^6 \text{sec} \times 5.4 \text{MHz} = 16.2 \text{ cycles}$ +14 cycle conversion = 30.2 -> 32 cycles. Then by looking in the User's guide, configured with remaining controls to the specified instructions in the comments. Using the upper range for the RI and CI values.

Table 5-24

RI ,Input MUX ON resistance= 4kohms

CI ,input capacitance= 15pF

RE, 10kohms

CE, 1pF

n=12

8.1 Code:

```
#include <msp430fr6989.h>
#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag
#define TXFLAG UCTXIFG // Transmit flag
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer
#define BUT1 BIT1 // P1.1
#define BUT2 BIT2 // P1.2
#define redLED BIT0 // Red at P1.0
#define greenLED BIT7 // Green at P9.7
unsigned int ascii[10] = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57};
void Initialize_ADC();
void Initialize_UART();
void config_ACLK_to_32KHz_crystal();
```

```

void uart_write_char(unsigned char ch);
void uart_write_string(char string[]);
void uart_write_uint16(unsigned int num);
unsigned int get_numSize(unsigned int num);
void parse_int_into_array(unsigned int num, unsigned int numSize, unsigned int* digits);
int main(void)
{
    WDTCTL = WDTPW | WDTHOLD; // stop watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // Enable GPIO pins
    //LED setup
    P1DIR |= redLED; // Pins as output
    P9DIR |= greenLED;
    P1OUT |= redLED; // Red on
    P9OUT &= ~greenLED; // Green off
    // Configure Channel 0 for up mode with interrupts
    TA0CCR0 = 32767; // 1 second @ 32 KHz
    TA0CCTL0 |= CCIE; // Enable Channel 0 CCIE bit
    TA0CCTL0 &= ~CCIFG; // Clear Channel 0 CCIFG bit
    // Timer_A: ACLK, div by 1, up mode, clear TAR
    TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLRL;
    TA0CTL &= ~TAIFG;
    // Enable the global interrupt bit (call an intrinsic function)
    _enable_interrupts();
    config_ACLK_to_32KHz_crystal();
    Initialize_ADC();
    Initialize_UART();
    unsigned int data = 0;
    while (1){
        _delay_cycles(500000);
        ADC12CTL0 |= ADC12SC;
        while ((ADC12CTL1 & ADC12BUSY) == ADC12BUSY){}
        data = ADC12MEM0;
        uart_write_uint16(data);
        uart_write_char("\n");
        uart_write_char("\r");
        data = 0;
    }
}

void Initialize_ADC() {
    // Divert the pins to analog functionality
    // X-axis: A10/P9.2, for A10 (P9DIR=x, P9SEL1=1, P9SEL0=1)
    P9SEL1 |= BIT2;
    P9SEL0 |= BIT2;
    // Y-axis: A4/P8.7, for A4 (P8DIR=x, P8SEL1=1, P8SEL0=1)
    P8SEL1 |= BIT7;
    P8SEL0 |= BIT7;
    // Turn on the ADC module
    ADC12CTL0 |= ADC12ON;
    // Turn off ENC (Enable Conversion) bit while modifying the configuration
    ADC12CTL0 &= ~ADC12ENC;
    //***** ADC12CTL0 *****
    // Set ADC12SHT0 (select the number of cycles that you determined)
    //32
    ADC12CTL0 |= ADC12SHT0_3;

```

```

//***** ADC12CTL1 *****
// Set ADC12SHS (select ADC12SC bit as the trigger)
//0
ADC12CTL1 |= ADC12SHS_0;
// Set ADC12SHP bit
//sample timer ,1
ADC12CTL1 |= ADC12SHP;
// Set ADC12DIV (select the divider you determined)//div 0
ADC12CTL1 |= ADC12DIV_0;
// Set ADC12SSEL (select MODOSC)
ADC12CTL1 |= ADC12SSEL_0;
//***** ADC12CTL2 *****
// Set ADC12RES (select 12-bit resolution)
ADC12CTL2 |= ADC12RES_2;//12 bit -> 14 cycles conversion
// Set ADC12DF (select unsigned binary format)
ADC12CTL2 &= ~ADC12DF;
//***** ADC12CTL3 *****
// Leave all fields at default values
//***** ADC12MCTL0 *****
// Set ADC12VRSEL (select VR+=AVCC, VR-=AVSS)
ADC12MCTL0 |= ADC12VRSEL_0;
// Set ADC12INCH (select channel A10)
ADC12MCTL0 |= ADC12INCH_10;
// Turn on ENC (Enable Conversion) bit at the end of the configuration
ADC12CTL0 |= ADC12ENC;
return;
}

void config_ACLK_to_32KHz_crystal() {
// By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
// Reroute pins to LFXIN/LFXOUT functionality
PJSEL1 &= ~BIT4;
PJSEL0 |= BIT4;
// Wait until the oscillator fault flags remain cleared
CSCTL0 = CSKEY; // Unlock CS registers
do {
CSCTL5 &= ~LFXTOFFG; // Local fault flag
SFRIFG1 &= ~OFIFG; // Global fault flag
} while((CSCTL5 & LFXTOFFG) != 0);
CSCTL0_H = 0; // Lock CS registers
return;
}

void Initialize_UART(){
// Configure pins to UART functionality
P3SEL1 &= ~(BIT4|BIT5);
P3SEL0 |= (BIT4|BIT5);
// Main configuration register
UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
// Most fields in this register, when set to zero, correspond to the popular configuration
UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
// Configure the clock dividers and modulators (and enable oversampling)
UCA1BRW = 6; // divider
// Modulators: UCBRF = 8 = 1000 --> UCBRF3 (bit #3)
// UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;

```

```

// Exit the reset state
UCA1CTLW0 &= ~UCSWRST;
return;
}
void uart_write_char(unsigned char ch){
// Wait for any ongoing transmission to complete
while ( (FLAGS & TXFLAG)==0 ) {}
// Copy the byte to the transmit buffer
TXBUFFER = ch; // Tx flag goes to 0 and Tx begins!
return;
}
void uart_write_string(char string[]){
int i = 0;
while(string[i] != '\0'){
    uart_write_char(string[i]);
    i++;
}
uart_write_char(string[i]);
}
void uart_write_uint16(unsigned int n){
int i;
    unsigned int numSize = 1; // Initialize numSize to 1
    unsigned int num = n; // Create a copy of n to avoid modifying the original value
    // Count number of digits
    while (num /= 10) {
        numSize++;
    }
    unsigned int digits[5]; // Number will not have more than 5 digits
    // Parse the numbers digit, place into declared array
    for (i = numSize - 1; i >= 0; i--) {
        digits[i] = n % 10;
        n = n / 10;
    }
    // Write each digit over UART
    for (i = 0; i < numSize; i++) {
        uart_write_char(ascii[digits[i]]);
    }
    // uart_write_char(10); // Write /n
    //uart_write_char(13); // Write /r
}
#pragma vector = TIMER0_A0_VECTOR
__interrupt void T0A0_ISR() {
    TA0CTL &= ~TAIFG;
    P1OUT ^= redLED;
}

```

8.2:

The next objective is to modify the previous code to convert the x and y coordinate signals. In main, we set up an infinite loop to do the conversion and send the result to UART, the same way the previous code did. The results are then read from the registers ADC12MEM0-1 to display on the UART terminal. The position value of both the X and Y axis of the joystick is displayed.

8.2 Code:

```
#include <msp430fr6989.h>
#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag
#define TXFLAG UCTXIFG // Transmit flag
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer
#define BUT1 BIT1 // P1.1
#define BUT2 BIT2 // P1.2
#define redLED BIT0 // Red at P1.0
#define greenLED BIT7 // Green at P9.7
unsigned int ascii[10] = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57};
void Initialize_ADC();
void Initialize_UART();
void config_ACLK_to_32KHz_crystal();
void uart_write_char(unsigned char ch);
void uart_write_string(char string[]);
void uart_write_uint16(unsigned int num);
int main(void)
{
    WDTCTL = WDTPW | WDTHOLD; // stop watchdog timer
    PM5CTL0 &= ~LOCKLPM5; // Enable GPIO pins
    //LED setup
    P1DIR |= redLED; // Pins as output
    P9DIR |= greenLED;
    P1OUT |= redLED; // Red on
    P9OUT &= ~greenLED; // Green off
    // Configure Channel 0 for up mode with interrupts
    TA0CCR0 = 32767; // 1 second @ 32 KHz
    TA0CCTL0 |= CCIE; // Enable Channel 0 CCIE bit
    TA0CCTL0 &= ~CCIFG; // Clear Channel 0 CCIFG bit
    // Timer_A: ACLK, div by 1, up mode, clear TAR
    TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLRL;
    TA0CTL &= ~TAIFG;
    // Enable the global interrupt bit (call an intrinsic function)
    _enable_interrupts();
    config_ACLK_to_32KHz_crystal();
    Initialize_ADC();
    Initialize_UART();
    unsigned int xAxis = 0;
    unsigned int yAxis = 0;
    while (1){
        delay_cycles(500000);
        ADC12CTL0 |= ADC12SC; //start conversion
        while ((ADC12CTL1 & ADC12BUSY) == ADC12BUSY){}
        xAxis = ADC12MEM0;
        yAxis = ADC12MEM1;
        uart_write_string("X-Axis: ");
        uart_write_uint16(xAxis);
        uart_write_string(" Y-Axis: ");
        uart_write_uint16(yAxis);
    }
}
```

```

    uart_write_char('\n');
    uart_write_char('\r');
    xAxis = 0;
    yAxis = 0;
}
}
void Initialize_ADC() {
// Divert the pins to analog functionality
// X-axis: A10/P9.2, for A10 (P9DIR=x, P9SEL1=1, P9SEL0=1)
P9SEL1 |= BIT2;
P9SEL0 |= BIT2;
// Y-axis: A4/P8.7, for A4 (P8DIR=x, P8SEL1=1, P8SEL0=1)
P8SEL1 |= BIT7;
P8SEL0 |= BIT7;
// Turn on the ADC module
ADC12CTL0 |= ADC12ON;
// Turn off ENC (Enable Conversion) bit while modifying the configuration
ADC12CTL0 &= ~ADC12ENC;
//***** ADC12CTL0 *****
// Set ADC12SHT0 (select the number of cycles that you determined)
//ADC12SHT0 = 32;
ADC12CTL0 |= ADC12SHT0_3;
// Set the bit ADC12MSC (Multiple Sample and Conversion)
ADC12CTL0 |= ADC12MSC;
//***** ADC12CTL1 *****
// Set ADC12SHS (select ADC12SC bit as the trigger)
//ADC12SHS = 0;
ADC12CTL1 |= ADC12SHS_0;
// Set ADC12SHP bit
//ADC12SHP = 1;
ADC12CTL1 |= ADC12SHP;
// Set ADC12DIV (select the divider you determined)
ADC12CTL1 |= ADC12DIV_0;
// Set ADC12SSEL (select MODOSC)
ADC12CTL1 |= ADC12SSEL_0;
// Set ADC12CONSEQ (select sequence-of-channels)
ADC12CTL1 |= ADC12CONSEQ_1;
//***** ADC12CTL2 *****
// Set ADC12RES (select 12-bit resolution)
ADC12CTL2 |= ADC12RES_2;
// Set ADC12DF (select unsigned binary format)
ADC12CTL2 &= ~ADC12DF;
//***** ADC12CTL3 *****
// Set ADC12CSTARTADD to 0 (first conversion in ADC12MEM0)
ADC12CTL3 |= ADC12CSTARTADD_0;
//***** ADC12MCTL0 *****
// Set ADC12VRSEL (select VR+=AVCC, VR-=AVSS)
ADC12MCTL0 |= ADC12VRSEL_0;
// Set ADC12INCH (select channel A10)
ADC12MCTL0 |= ADC12INCH_10;
//***** ADC12MCTL1 *****
// Set ADC12VRSEL (select VR+=AVCC, VR-=AVSS)
ADC12MCTL1 |= ADC12VRSEL_0;
// Set ADC12INCH (select the analog channel that you found)

```

```

ADC12MCTL1 |= ADC12INCH_4;
// Set ADC12EOS (last conversion in ADC12MEM1)
ADC12MCTL1 |= ADC12EOS;
// Turn on ENC (Enable Conversion) bit at the end of the configuration
ADC12CTL0 |= ADC12ENC;
return;
}
void config_ACLK_to_32KHz_crystal() {
// By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
// Reroute pins to LFXIN/LFXOUT functionality
PJSEL1 &= ~BIT4;
PJSEL0 |= BIT4;
// Wait until the oscillator fault flags remain cleared
CSCTL0 = CSKEY; // Unlock CS registers
do {
CSCTL5 &= ~LFXTOFFG; // Local fault flag
SFRIFG1 &= ~OFIFG; // Global fault flag
} while((CSCTL5 & LFXTOFFG) != 0);
CSCTL0_H = 0; // Lock CS registers
return;
}
void Initialize_UART(){
// Configure pins to UART functionality
P3SEL1 &= ~(BIT4|BIT5);
P3SEL0 |= (BIT4|BIT5);
// Main configuration register
UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
// Most fields in this register, when set to zero, correspond to the popular configuration
UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
// Configure the clock dividers and modulators (and enable oversampling)
UCA1BRW = 6; // divider
// Modulators: UCBRF = 8 = 1000 -> UCBRF3 (bit #3)
// UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;
// Exit the reset state
UCA1CTLW0 &= ~UCSWRST;
return;
}
void uart_write_char(unsigned char ch){
// Wait for any ongoing transmission to complete
while ( (FLAGS & TXFLAG)==0 ) {}
// Copy the byte to the transmit buffer
TXBUFFER = ch; // Tx flag goes to 0 and Tx begins!
return;
}
void uart_write_string(char string[]){
int i = 0;
while(string[i] != '\0'){
uart_write_char(string[i]);
i++;
}
uart_write_char(string[i]);
}
void uart_write_uint16(unsigned int n){

```

```

int i;
    unsigned int numSize = 1; // Initialize numSize to 1
    unsigned int num = n; // Create a copy of n to avoid modifying the original value
    // Count number of digits
    while (num /= 10) {
        numSize++;
    }
    unsigned int digits[5]; // Number will not have more than 5 digits
    // Parse the numbers digit, place into declared array
    for (i = numSize - 1; i >= 0; i--) {
        digits[i] = n % 10;
        n = n / 10;
    }
    // Write each digit over UART
    for (i = 0; i < numSize; i++) {
        uart_write_char(ascii[digits[i]]);
    }
    // uart_write_char(10); // Write /n
    //uart_write_char(13); // Write /r
}
#pragma vector = TIMER0_A0_VECTOR
__interrupt void T0A0_ISR() {
    TA0CTL &= ~TAIFG;
    P1OUT ^= redLED;
}

```

Conclusion:

This lab demonstrated the practical use of the Analog-to-Digital Converter (ADC) module to interface with a 2-dimensional joystick. By configuring the ADC, we successfully captured joystick movement data and reconfigured it to obtain accurate X and Y coordinates. Providing skills for programming embedded systems with analog signal interfacing.

Student Q&A

1. How many cycles does it take the ADC to convert a 12-bit result? (look in the configuration register that contains ADC12RES).

14 cycles

2. In this experiment, we set our reference voltages $V_{R+} = V_{CC}$ (Analog V_{CC}) and $V_{R-} = V_{SS}$ (Analog V_{SS}). What voltage values do these signals have? Look in the MCU data sheet (slas789c) in Table 5.3. Assume that $V_{CC}=3.3V$ and $V_{SS}=0$.

$V_{CC}= 1.8 -3.6 V$